5주차 공부 내용 정리

교재 6-7장 | AI Agent 프레임워크 비교, Colab 환경 구성과 API 키 설정 | 팀 알아서할게

5주차에는 교재 6장에서 AutoGen, LangChain, LangGraph, CrewAI, LlamaIndex, AutoGPT를 비교하고, 7장을 따라 실습할 Colab 환경과 API 키를 준비했다. 6주차에 같은 에이전트를 프레임워크마다 구현해 볼 예정이라서 각 프레임워크가 무엇을 기본 단위로 삼는지에 맞춰 정리했다.

01 프레임워크 개요

여섯 프레임워크는 모두 LLM 호출, 도구, 메모리를 묶어 에이전트를 만드는 도구지만 기본 단위가 다르다. LangChain은 구성요소를 잇는 체인, LangGraph는 상태가 흐르는 그래프, AutoGen과 CrewAI는 역할을 가진 에이전트, LlamaIndex는 검색할 데이터의 인덱스를 중심에 둔다. AutoGPT는 목표만 주면 스스로 반복 실행하는 완성형 에이전트에서 출발했다는 점이 나머지와 다르다.

프레임워크	만든 곳과 공개 시기	기본 단위	잘 맞는 작업
LangChain	Harrison Chase, 2022년 10월	모델, 프롬프트, 도구를 잇는 체인	LLM 앱의 기본 구성, RAG, 단순한 도구 호출 에이전트
LangGraph	LangChain 팀, 2024년 1월	노드와 엣지로 된 상태 그래프	분기와 반복이 있는 흐름, 사람 승인 단계가 필요한 작업
AutoGen	Microsoft Research, 2023년	대화하는 에이전트	에이전트끼리 대화하며 코드를 쓰고 실행하는 작업
CrewAI	João Moura, 2023년 말	역할, 목표, 작업을 가진 크루	조사, 작성, 검토처럼 역할 분담이 분명한 작업
LlamaIndex	Jerry Liu, 2022년 11월	문서를 담은 인덱스와 쿼리 엔진	사내 문서 검색처럼 데이터 연결이 중요한 RAG
AutoGPT	Significant Gravitas, 2023년 3월	목표를 받아 스스로 반복하는 에이전트	자율 실행 구조를 이해하는 실험, 블록 기반 자동화

02 LangChain

LangChain은 LLM 앱에 필요한 구성요소를 같은 인터페이스로 제공하는 프레임워크다. 채팅 모델, 프롬프트 템플릿, 출력 파서, 문서 로더, 텍스트 분할기, 벡터 저장소, 리트리버, 도구가 모두 `invoke`로 실행되고, 모델 제공사를 바꿀 때 `ChatOpenAI`를 `ChatAnthropic`으로 바꾸는 정도만 고치면 된다.

LCEL로 체인 만들기

LangChain Expression Language(LCEL)는 구성요소를 `|` 연산자로 이어 체인을 만드는 문법이다. 앞 구성요소의 출력이 다음 구성요소의 입력으로 넘어간다.

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

prompt = ChatPromptTemplate.from_template("{topic}을 한 문장으로 설명해 줘")
llm = ChatOpenAI(model="gpt-4o-mini")
chain = prompt ¦ llm ¦ StrOutputParser()
print(chain.invoke({"topic": "AI Agent"}))
```

2025년 10월에 나온 LangChain 1.0은 에이전트를 만드는 함수를 `create_agent`로 정리했고, 이 함수는 내부에서 LangGraph로 실행된다. 그래서 단순한 에이전트는 LangChain으로 만들고, 흐름을 직접 설계해야 하면 LangGraph로 내려가는 방식으로 두 프레임워크를 함께 쓴다. 7주차에 볼 LangSmith도 같은 회사가 만든 추적과 평가 도구다.

03 LangGraph

LangGraph는 에이전트의 흐름을 그래프로 정의하는 프레임워크다. 체인은 한 방향으로만 흐르지만, 에이전트는 도구 결과를 보고 앞 단계로 돌아가거나 조건에 따라 다른 단계로 가야 해서 그래프 구조가 필요하다. 4주차에 본 반응 패턴과 계획 패턴의 반복 구조를 그대로 옮길 수 있다.

구성요소	역할
State	그래프 전체가 공유하는 데이터다. 노드는 State를 읽고 바꿀 값만 돌려준다.
Node	LLM 호출, 도구 실행 같은 작업 하나를 맡는 함수다.
Edge	다음에 실행할 노드를 정한다. 조건부 엣지는 State 값을 보고 갈 곳을 고른다.
Checkpointer	단계마다 State를 저장해서 대화를 이어 가거나 중간에 멈췄다가 다시 시작할 수 있게 한다.

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class State(TypedDict):
    question: str
    answer: str

def answer_node(state: State):
    return {"answer": llm.invoke(state["question"]).content}

graph = StateGraph(State)
graph.add_node("answer", answer_node)
graph.add_edge(START, "answer")
graph.add_edge("answer", END)
app = graph.compile()
app.invoke({"question": "LangGraph는 무엇인가?"})
```

Checkpointer가 있으면 `interrupt`로 특정 노드 앞에서 실행을 멈추고 사람의 승인을 받은 뒤 이어 갈 수 있다. 결제나 메일 발송처럼 되돌리기 어려운 행동 앞에 승인 단계를 두는 데 쓴다.

04 AutoGen

AutoGen은 Microsoft Research가 2023년에 공개한 멀티에이전트 대화 프레임워크다. 에이전트끼리 메시지를 주고받으며 작업을 진행하는데, 대표적인 구성은 LLM이 코드를 쓰면 사용자 대리 에이전트가 그 코드를 실행하고 결과를 다시 돌려주는 두 에이전트 대화다.

작업 요청 → Assistant가 코드 작성 → 실행 에이전트가 코드 실행 → 결과 전달 → 종료 조건 확인

오류가 나면 수정한 코드로 반복

2025년 1월에 나온 0.4 버전은 구조를 비동기 이벤트 기반으로 다시 만들고 Core, AgentChat, Extensions로 나눴다. AgentChat에는 에이전트가 돌아가며 말하는 `RoundRobinGroupChat`, LLM이 다음 발언자를 고르는 `SelectorGroupChat` 같은 팀 구성이 있고, 코드 없이 에이전트를 조립하는 AutoGen Studio도 있다.

> AutoGen은 버전과 프로젝트가 여러 갈래로 나뉘어 있어서 교재 코드를 따라 할 때 버전을 확인해야 한다. 0.2와 0.4는 API가 다르고, 2024년 말에는 원래 개발진 일부가 AG2라는 이름으로 0.2 계열을 따로 이어 갔다. Microsoft는 2025년 10월에 AutoGen과 Semantic Kernel을 합친 Microsoft Agent Framework를 발표했고, AutoGen은 버그와 보안 수정만 하는 유지 관리 단계로 넘어갔다.

05 CrewAI

CrewAI는 회사 조직처럼 역할을 나눈 에이전트 팀을 구성하는 프레임워크다. 에이전트마다 역할, 목표, 배경을 자연어로 주고, 작업마다 설명과 기대 결과를 정한 뒤 크루로 묶어 실행한다. 초기에는 LangChain 위에서 만들어졌지만 지금은 LangChain 없이 독립적으로 동작한다.

구성요소	설정하는 내용
Agent	`role`, `goal`, `backstory`와 사용할 도구
Task	작업 설명, 기대 결과(`expected_output`), 맡을 에이전트
Crew	에이전트와 작업 목록, 실행 방식
Process	작업을 순서대로 넘기는 sequential, 관리자 에이전트가 작업을 배분하고 검토하는 hierarchical

```
from crewai import Agent, Task, Crew, Process

researcher = Agent(role="조사원", goal="AI Agent 프레임워크 동향을 찾는다", backstory="기술 동향 분석가")
writer = Agent(role="작성자", goal="조사 결과를 보고서로 쓴다", backstory="기술 블로그 작성자")
research = Task(description="프레임워크 동향 조사", expected_output="동향 5개", agent=researcher)
report = Task(description="조사 결과로 보고서 작성", expected_output="한 쪽 보고서", agent=writer)

crew = Crew(agents=[researcher, writer], tasks=[research, report], process=Process.sequential)
crew.kickoff()
```

설정이 자연어 위주라서 빨리 만들 수 있는 대신, 에이전트 사이에 어떤 내용이 오가는지 세밀하게 제어하기는 LangGraph보다 어렵다. 이런 한계를 보완하려고 이벤트에 따라 단계를 잇는 Flows 기능이 추가됐다.

06 LlamaIndex

LlamaIndex는 LLM이 외부 데이터를 검색해 답하도록 연결하는 데이터 프레임워크다. 2022년 11월에 GPT Index라는 이름으로 시작했고, 2주차에 비교한 RAG를 만드는 데 가장 많이 쓰인다.

문서 불러오기 → 노드로 분할 → 임베딩과 인덱스 생성 → 관련 노드 검색 → LLM이 답 생성

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader

docs = SimpleDirectoryReader("data").load_data()
index = VectorStoreIndex.from_documents(docs)
print(index.as_query_engine().query("휴가 신청 절차는?"))
```

PDF, 노션, 데이터베이스 같은 데이터를 불러오는 연결기는 LlamaHub에 모여 있다. 쿼리 엔진을 도구로 감싸 에이전트에게 주면 에이전트가 질문에 따라 어떤 문서 묶음을 검색할지 고를 수 있고, 여러 단계를 이벤트로 잇는 Workflows로 더 복잡한 에이전트를 만들 수 있다.

07 AutoGPT

AutoGPT는 2023년 3월에 공개된 자율 에이전트로, 이름과 역할, 목표만 입력하면 GPT-4가 할 일을 정하고 검색, 파일 쓰기, 코드 실행을 반복하며 목표를 처리한다. 공개 직후 GitHub에서 가장 빠르게 별을 모은 저장소 중 하나가 되면서 AI Agent라는 말을 널리 알렸다.

목표 입력 → 다음 할 일 생각 → 명령 실행 → 결과를 메모리에 저장 → 목표 달성 판단

달성하지 못하면 다시 생각부터 반복

실제로 써 보면 같은 행동을 반복하거나 목표와 관계없는 작업으로 빠지는 일이 잦았고, 반복할 때마다 GPT-4를 호출해서 비용이 많이 들었다. 이 경험 때문에 이후 프레임워크들은 사람이 흐름을 설계하고 LLM은 정해진 범위 안에서 판단하게 하는 쪽으로 발전했다. 지금의 AutoGPT는 블록을 이어 에이전트를 만드는 AutoGPT Platform이 중심이고, 처음의 자율 실행 버전은 Classic으로 남아 있다.

08 Colab 환경 구성과 API 키 설정

교재 실습은 Google Colab에서 진행한다. Colab은 브라우저에서 바로 Python 노트북을 실행할 수 있어 팀원마다 설치 환경이 달라서 생기는 문제를 줄인다.

순서	할 일	내용
01	패키지 설치	`!pip install -q langchain langchain-openai langgraph` 처럼 셀에서 설치한다. 런타임이 초기화되면 다시 설치해야 한다.
02	API 키 발급	OpenAI 플랫폼의 API keys 메뉴에서 키를 만든다. 키는 만들 때 한 번만 보여 주고, 사용량만큼 요금이 나가므로 결제 수단과 사용 한도를 함께 설정한다.
03	Secrets에 저장	Colab 왼쪽 사이드바의 열쇠 아이콘에서 `OPENAI_API_KEY` 라는 이름으로 키를 저장하고 노트북 액세스를 켠다.

04	환경 변수 설정	아래 코드로 저장한 키를 환경 변수에 넣으면 LangChain, CrewAI 같은 라이브러리가 자동으로 읽는다.

```
import os
from google.colab import userdata

os.environ["OPENAI_API_KEY"] = userdata.get("OPENAI_API_KEY")
```

키를 코드 셀에 직접 적으면 노트북을 공유하거나 GitHub에 올릴 때 키가 함께 노출된다. 노출된 키는 다른 사람이 쓸 수 있어서 바로 폐기하고 새로 발급해야 한다. 로컬에서 실행할 때는 `.env` 파일에 키를 두고 `python-dotenv` 로 읽으며, `.env` 는 `.gitignore` 에 넣는다.

여러 프레임워크를 한 노트북에 모두 설치하면 공통으로 쓰는 패키지의 버전 요구가 달라 충돌할 수 있다. 프레임워크마다 노트북을 나누고, 교재와 다른 버전이 설치되면 `pip install langchain==버전` 처럼 교재가 쓴 버전으로 맞추는 편이 안전하다.

09 더 생각해 볼 질문

- 프레임워크를 고르는 기준은 무엇이어야 하는가? 흐름을 세밀하게 제어해야 하면 LangGraph, 역할 분담을 빨리 시험해 보려면 CrewAI, 문서 검색이 중심이면 LlamaIndex가 맞아서, 작업의 성격을 먼저 정해야 한다.
- 프레임워크 없이 API만으로 만들면 어떤가? 단순한 도구 호출 에이전트는 반복문 몇 줄로도 만들 수 있고, 프레임워크의 추상화가 오류 원인을 찾기 어렵게 만든다는 지적도 있어서 6주차 구현에서 직접 비교해 볼 만하다.

참고 자료: 서지영, 랭체인 & 랭그래프로 AI 에이전트 개발하기, 길벗 / LangChain 공식 문서, python.langchain.com / LangGraph 공식 문서, langchain-ai.github.io/langgraph / LangChain, LangChain and LangGraph 1.0, 2025 / Wu et al., AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation, 2023 / Microsoft, AutoGen v0.4 발표, 2025 / Microsoft, Introducing Microsoft Agent Framework, 2025 / CrewAI 공식 문서, docs.crewai.com / LlamaIndex 공식 문서, docs.llamaindex.ai / Significant-Gravitas, AutoGPT GitHub 저장소 / Google Colab, Secrets 사용 안내